

PMPH

Assignment 2 Report

Bora Haliloglu

September 2026

1 Task 1

1.1 Implementation

The irregular nested computation of composite numbers was transformed into a flat-parallel implementation using `scan`, `scatter`, and `map`.

For each prime, the number of generated multiples is computed as

$$\text{mult_lens}[p] = \left\lfloor \frac{\text{len}}{p} \right\rfloor - 1.$$

The total length of the flattened composite array is

$$\text{flat_size} = \sum_p \text{mult_lens}[p].$$

The implemented flattening is:

```
let mult_lens = map (\p -> (len / p) - 1) sq_primes
let flat_size = reduce (+) 0 mult_lens

let ends = scan (+) 0 mult_lens
let starts = map2 (-) ends mult_lens

let insert_start =
  scatter (replicate flat_size 0i64)
    starts
    sq_primes

let flat_prime =
  scan (\z p -> if p > z then p else z)
    0i64
    insert_start

let segment_end =
```

```

scatter (replicate flat_size 0i64)
      starts
      starts

let segment_starts =
  scan (\z p -> if p > z then p else z)
    0i64
    segment_end

let not_primes =
  map (\i ->
    let prime = flat_prime[i]
    let start_of_segment = segment_starts[i]
    in (i - start_of_segment + 2) * prime)
    (iota flat_size)

```

A prefix sum over `mult_lens` determines the segment boundaries. The corresponding prime and segment-start index are scattered to the segment beginnings and propagated through the flat array using prefix scans.

For a global flat index i belonging to a segment starting at s , the composite value is computed as

$$(i - s + 2)p.$$

This reproduces the nested sequence

$$2p, 3p, \dots, \left\lfloor \frac{\text{len}}{p} \right\rfloor p$$

for each prime p , without constructing an irregular nested array. Thus, the implementation preserves the work-efficient sieve structure while exposing flat parallelism suitable for GPU execution.

1.2 Validation

Correctness was tested using

```
futhark test --backend=cuda primes-flat.fut
```

and both provided datasets passed:

- $n = 30$, producing

$$[2, 3, 5, 7, 11, 13, 17, 19, 23, 29],$$

- $n = 10\,000\,000$, validated against `ref10000000.out`.

Therefore, the flat-parallel implementation produces the expected result for both the small and large datasets.

1.3 Performance

Implementation	Runtime (μs)
<code>primes-seq.fut</code>	184 270
<code>primes-naive.fut</code>	64 903
<code>primes-adhoc.fut</code>	183 601
<code>primes-flat.fut</code>	2 140

Table 1: Runtime for prime computation with $n = 10\,000\,000$.

For the large dataset, the flat implementation achieved speedups of approximately

$$\frac{184270}{2140} \approx 86.1\times$$

over the sequential implementation,

$$\frac{64903}{2140} \approx 30.3\times$$

over the naive implementation, and

$$\frac{183601}{2140} \approx 85.8\times$$

over the ad-hoc implementation.

The sequential implementation performs the computation without exploiting GPU parallelism. The naive implementation exposes parallelism, but its sieve structure has higher sequential depth. The ad-hoc implementation provides regular parallelism but performs additional work.

The flat implementation preserves the work-efficient structure of the nested sieve while replacing irregular nested parallelism with flat operations such as scans, scatters, and maps. It combines low parallel depth with efficient work complexity and achieves substantially better performance on the large dataset.

For $n = 30$, the measured runtimes were $2\,\mu s$, $165\,\mu s$, $285\,\mu s$, and $215\,\mu s$ for the sequential, naive, ad-hoc, and flat implementations, respectively. At this small problem size, GPU kernel-launch and synchronization overhead dominate the computation, so the sequential CPU implementation is faster despite its lack of parallelism.

2 Task 2

2.1 Implementation

This task was to improve the global memory access pattern. In the original implementation each thread accessed consecutive elements assigned to itself:

```
uint32_t loc_ind = threadIdx.x * CHUNK + i;
```

This causes neighboring threads in a warp to access memory locations separated by `CHUNK` elements. This access pattern prevents efficient memory coalescing.

The indexing was changed to:

```
#if 1
    uint32_t loc_ind =
        i * blockDim.x * blockDim.y * blockDim.z
        + threadIdx.x;
#else
    uint32_t loc_ind =
        threadIdx.x * CHUNK + i;
#endif
```

For the kernels used in this assignment, the thread blocks are one-dimensional. Therefore,

$$\text{blockDim.y} = \text{blockDim.z} = 1,$$

and the new expression is equivalent to

$$\text{loc_ind} = i \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

With this layout, consecutive threads access consecutive elements. For example, with a block size of 256, the first iteration accesses

$$0, 1, 2, \dots, 255,$$

while the second iteration accesses

$$256, 257, 258, \dots, 511.$$

Threads belonging to the same warp access adjacent memory locations, allowing the GPU to combine their accesses into coalesced global memory transactions.

2.2 Experimental Setup

The optimized implementation was tested on an NVIDIA A100-PCIE-40GB GPU. The main test was executed using

```
./test_pbb 10000000 256
```

with the following parameters:

- Number of elements: $N = 10,000,000$
- Block size: 256
- Elements per thread: 24

- GPU benchmark repetitions: 1000

All reduction and scan implementations were also compared against their sequential implementations in order to verify correctness.

2.3 Results

Table 2 shows the measured GPU execution times and effective memory bandwidths after applying the coalesced indexing scheme.

Operation	Time (μ s)	Bandwidth (GB/s)
Naive Memory Copy	62	1290.32
Naive Reduce – Int32	177	225.99
Optimized Reduce – Int32	48	833.33
Naive Reduce – MSSP	481	83.16
Optimized Reduce – MSSP	110	363.64
Inclusive Scan – Int32	147	816.33
Segmented Scan – Int32	173	809.25

Table 2: Performance after applying coalesced global-memory indexing.

All parallel basic-block tests produced correct results:

```
Reduce: VALID result!
Reduce: VALID result!
Reduce: VALID result!
Reduce: VALID result!
Scan Inclusive AddI32: VALID result!
SgmScan Inclusive AddI32: VALID result!
```

This confirms that changing the indexing scheme preserved the semantics of the algorithms.

The optimized integer reduction achieved a bandwidth of

833.33 GB/s,

while the inclusive and segmented scans achieved

816.33 GB/s

and

809.25 GB/s,

respectively. The simple memory-copy kernel reached approximately

1290.32 GB/s.

These results show that the kernels can make significantly better use of the available global-memory bandwidth when consecutive threads access consecutive memory addresses.

2.4 Discussion

The original indexing,

$$\text{loc_ind} = \text{threadIdx.x} \cdot \text{CHUNK} + i,$$

causes neighboring threads to access memory locations separated by `CHUNK`, resulting in inefficient global-memory transactions.

The modified indexing,

$$\text{loc_ind} = i \cdot \text{blockDim.x} + \text{threadIdx.x},$$

makes neighboring threads access consecutive elements, enabling coalesced memory accesses.

Using coalesced global memory accesses improved GPU performance while preserving correctness. Several kernels achieved more than 800 GB/s of effective memory bandwidth, demonstrating the importance of contiguous memory access patterns for CUDA threads.

3 Task 3

The warp-level inclusive scan was re-implemented using a parallel scan inside each warp. The implementation performs $\log_2(\text{WARP})$ steps. At step d , each lane with index at least 2^d combines its current value with the value stored 2^d positions before it.

```
#pragma unroll
for (int d = 0; d < k; d++) {
    int h = pow(2.0, d);
    if (lane >= h) {
        ptr[idx] = OP::apply(ptr[idx-h], ptr[idx]);
    }
}
```

This replaces the sequential version, where one thread performed the complete scan over the warp. The new implementation therefore exposes more parallelism and has a work depth of $O(\log(\text{WARP}))$ instead of $O(\text{WARP})$.

All tests produced **VALID** results, confirming that the implementation computes the correct inclusive scan.

Although the new implementation is more parallel algorithmically, the measured performance is lower than the old implementation for all tested input sizes. For $N = 10^6$, for example, the execution time increases from 24 μs to 35 μs , while throughput decreases from 500 GB/s to approximately 343 GB/s.

The result shows that increasing theoretical parallelism does not necessarily improve GPU performance. For a warp of only 32 threads, the additional shared-memory accesses, dependencies between scan stages, and control operations can outweigh the benefit of the logarithmic-depth scan. Nevertheless, the new implementation remains correct for all tested input sizes.

N	Old implementation		New implementation	
	Time (μ s)	GB/s	Time (μ s)	GB/s
1,000	31	0.39	33	0.36
10,000	18	6.67	32	3.75
100,000	19	63.16	25	48.00
1,000,000	24	500.00	35	342.86

Table 3: Performance of the inclusive AddI32 scan on an NVIDIA A100-PCIE-40GB.

4 Task 4

The original implementation of `scanIncBlock` contains a race condition when the block size is 1024. After the warp-level scan, the last thread of each warp stores the total result of its warp as

```
ptr[warpid] = OP::remVolatile(ptr[idx]);
```

For a block of 1024 threads, there are 32 warps. Therefore, warp 0 reads `ptr[31]` to obtain its final value, while warp 31 may simultaneously write its result to `ptr[31]`. Since these operations occur in different warps, their execution order is not guaranteed, producing a race condition.

The bug appears only for a block size of 1024 because this is the first case with 32 warps. For smaller blocks, the warp result locations `ptr[0..30]` do not overlap with the last element of warp 0, `ptr[31]`.

The problem was fixed by using the per thread result already returned by `scanIncWarp`:

```
if (lane == (WARP-1)) {
    ptr[warpid] = res;
}
```

Since `res` is stored privately by each thread, no shared-memory value has to be read while the warp totals are being written. This removes the read-write race. After the modification, the inclusive scan and segmented scan both produced valid results with a block size of 1024.

5 Task 5

The sparse matrix-vector multiplication was implemented on the GPU using a segmented-scan formulation. The sparse matrix is stored in flattened form by keeping the non-zero matrix values in `mat.vals` and their corresponding column indices in `mat.inds`. The row lengths are stored separately and are used to construct the segment boundaries.

The GPU implementation consists of four kernels. First, `replicate0` initializes the flag array to zero. Then, `mkFlags` marks the first element of every

matrix row. The row-start positions are obtained from the inclusive scan of the row-length array.

For example, if the row lengths are

$$[3, 2, 4],$$

their inclusive scan is

$$[3, 5, 9].$$

Hence, the corresponding row starts are

$$0, 3, 5,$$

which are marked in the flag array.

The `mult_pairs` kernel computes the contribution of every non-zero matrix element as

$$\text{tmp_pairs}[i] = \text{mat_vals}[i] \cdot \text{vct}[\text{mat_inds}[i]].$$

A segmented inclusive scan is then applied to these values using the previously constructed flag array. As a result, the last element of each segment contains the complete dot product of the corresponding matrix row.

Finally, `select_last_in_sgm` extracts these values using

$$\text{tmp_scan}[\text{mat_shp_sc_d}[i] - 1]$$

and stores them in the result vector.

The number of CUDA blocks was computed using ceiling division,

$$N_{\text{blocks}} = \left\lceil \frac{N}{B} \right\rceil = \frac{N + B - 1}{B},$$

where N is the number of elements processed by the kernel and B is the CUDA block size. Therefore, kernels operating on all non-zero matrix elements use `tot_size`, while kernels operating on matrix rows use `mat_rows`.

The implementation was tested on an NVIDIA A100-PCIE-40GB GPU with several input sizes and CUDA block sizes. The results are shown in Table 4.

Rows	Vector Size	Block Size	CPU (μs)	GPU (μs)	Speedup
11033	2076	1024	19612	493	$39.78\times$
11033	2076	256	19605	386	$50.79\times$
1000	500	512	427	53	$8.06\times$
10	20	32	< 1	73	—

Table 4: Sparse matrix–vector multiplication performance.

All tested configurations produced a valid result when compared with the sequential CPU implementation.

For the largest input, the best measured performance was obtained with a block size of 256, where the GPU execution time was 386 μs compared with 19605 μs on the CPU. This corresponds to a speedup of approximately $50.79\times$.

Using 1024 threads per block was slower, with a runtime of 493 μ s. This shows that using the maximum possible CUDA block size does not necessarily provide the best performance.

For the smallest input, the GPU execution time was higher than the CPU time. In this case, the computation is too small to effectively utilize the GPU, and kernel-launch and synchronization overhead dominate the execution time.

Overall, the segmented scan implementation produced correct results for all tested configurations and achieved a significant speedup for sufficiently large sparse matrices.